

I'd give Codex a specification like this. It defines the architecture, implementation phases, coding standards, and expected behavior without over-constraining the implementation.

Project: BlowAway

Objective

Develop a production-quality Android application called **BlowAway**.

The purpose of the app is to allow users to dismiss **Heads-Up Notification (HUN) popups** by blowing into the device microphone.

The notification should:

- disappear from the display
- remain inside the notification shade
- remain unread
- remain actionable
- not be cancelled
- not be removed from notification history

The app should only dismiss the temporary popup/banner.

Platform

Android only.

Language:

- Kotlin

UI:

- Jetpack Compose

Architecture:

- Clean Architecture
- MVVM

Dependency Injection:

- Hilt

Async:

- Kotlin Coroutines
- Flow

Persistence:

- Room
- DataStore

Testing:

- JUnit
- MockK
- Instrumentation tests

Minimum SDK:

29

Target SDK:

Latest stable Android SDK

Core Architecture

Settings UI



Permission Manager



NotificationListenerService



AccessibilityService



Notification Tracker

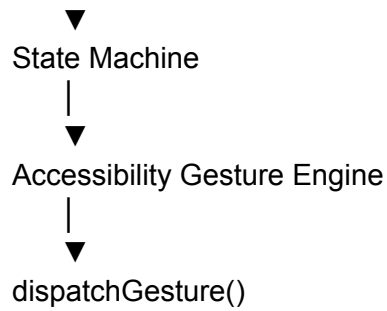


Foreground Microphone Service



Blow Detection Engine





Required Android Components

Implement:

- AccessibilityService
 - NotificationListenerService
 - ForegroundService
 - AudioRecord microphone capture
 - Jetpack Compose UI
 - DataStore
 - Room database
-

State Machine

Implement the following application state machine.

Idle



Heads-up notification detected



Notification Active



Listening For Blow



Blow Confirmed



Swipe Gesture Executed



Cooldown (2 seconds)



Idle

The microphone processing should only actively evaluate for blow gestures while a heads-up notification is active.

This minimizes battery usage and false positives.

Notification Tracking

Use `NotificationListenerService`.

When a notification appears:

Store:

- timestamp
- package name
- category
- importance
- notification key

Open a configurable active window.

Default:

3 seconds

If the notification disappears naturally before the blow occurs:

Return to Idle.

Accessibility Service

Responsibilities:

Observe accessibility events.

When possible:

Locate the current Heads-Up Notification using `AccessibilityNodeInfo`.

Determine:

- x
- y
- width
- height

Cache those bounds.

If bounds cannot be located:

Fall back to configurable default swipe coordinates.

When Blow Detection requests dismissal:

Execute

`dispatchGesture()`

Swipe vertically upward from the centre of the cached notification bounds.

Gesture duration:

150–250ms

Fallback Strategy

Priority:

1.

`AccessibilityNodeInfo` bounds

2.

Window metrics estimation

3.

User calibrated coordinates

Never hardcode device-specific coordinates.

Blow Detection Engine

Use AudioRecord.

Configuration:

16 kHz

PCM 16-bit

Mono

20 ms frames

Implement adaptive noise floor estimation.

Features to compute:

- RMS
- Peak amplitude
- Zero Crossing Rate
- Spectral centroid
- Spectral flatness
- Frame energy

Trigger requirements:

- adaptive threshold exceeded
- broadband airflow signature
- duration between 150–700 ms
- not clipping
- cooldown expired

Do not rely on amplitude alone.

Voice Rejection

Integrate Voice Activity Detection.

Speech should never trigger dismissal.

Reject:

- talking
 - TV
 - podcasts
 - phone calls
-

Phase 1

Implement heuristic detector.

Use:

Adaptive RMS

Zero Crossing Rate

Noise floor

Cooldown

Confidence score

Phase 2

Replace heuristic detector with TensorFlow Lite.

Train classifier.

Labels:

- Blow
- Speech
- Cough
- Wind
- Fan
- TV
- Music
- Keyboard

- Traffic
- Silence

Only Blow should trigger dismissal.

Calibration

Create calibration wizard.

User performs five blows.

Store:

Average RMS

Average duration

Peak amplitude

Spectral centroid

Detection thresholds should adapt to calibration.

Settings Screen

Implement:

General

- ☐ Enable BlowAway
- ☐ Start on boot
- ☐ Foreground service enabled

Detection

Sensitivity slider

Cooldown slider

Calibration button

Notification Behaviour

Listening window

Gesture duration

Fallback mode

Conditions

- ☐ Screen must be on
- ☐ Device unlocked only
- ☐ Ignore alarms
- ☐ Ignore phone calls
- ☐ Ignore navigation
- ☐ Ignore media
- ☐ Ignore DND
- ☐ Ignore charging

Per-app settings

Whitelist

Blacklist

Search

Icons

Debug Screen

Display live diagnostics.

Include:

Microphone waveform

Current RMS

Noise floor

Blow confidence

Speech confidence

Current application

Current notification

Accessibility bounds

Gesture coordinates

State machine state

Cooldown timer

Total dismissals

False triggers

Missed detections

Debug logging export

Gesture Engine

Create reusable gesture system.

Functions:

Swipe Up

Swipe Down

Swipe Left

Swipe Right

Tap

Long Press

Future gestures should require minimal code changes.

Battery Optimisation

Suspend microphone processing while:

No active notification

Screen off (optional)

User disabled feature

Use coroutines efficiently.

Avoid unnecessary allocations.

Privacy

All audio processing must occur on-device.

No microphone recordings may be stored.

No audio may leave the device.

No analytics.

No cloud processing.

UI

Use Material 3.

Support:

Light mode

Dark mode

Dynamic color

Responsive layouts

Animations should be subtle.

Code Quality

Requirements:

Well documented

Modular

SOLID principles

Clean Architecture

Dependency injection

Unit tested

Production ready

No deprecated Android APIs

No hardcoded constants

Meaningful logging

Proper error handling

Future Expansion

Design the architecture so future actions can be added.

Examples:

Blow → dismiss notification

Double blow → dismiss all popups

Whistle → open notification

Finger snap → mute ringtone

Voice keyword → custom action

The detection engine should be generic enough to support multiple trigger types.

Deliverables

Produce:

1. Complete Android Studio project
2. Buildable source code
3. Unit tests
4. Instrumentation tests
5. Documentation
6. Architecture diagram
7. Permission flow
8. Setup instructions
9. README
10. Inline code documentation
11. Suggested TensorFlow Lite training pipeline
12. Sample audio dataset structure
13. Production-ready Gradle configuration